Breadcrumbs Autonomous Rover

1. System Overview & Target Environment

- **Project Concept:** Offline, GPS-denied autonomous hiker companion and backtracking rover ("Hansel & Gretel" breadcrumb navigation architecture).
- **Compute & Middleware:** Raspberry Pi 5 (8GB RAM) running **ROS 2** (Ubuntu/Debian Linux, 100% offline/API-free execution).
- **Sensor & Actuator Stack:**
    - **2D LiDAR:** Youyeetoo D300 (LD19 series), UART interface at 230,400 baud, mapped via persistent udev rule (`10c4:ea60` → `/dev/lidar`) using `ldlidar_stl_ros2`.
    - **Drivetrain & Odometry:** 2-channel differential/tank drive motor controller with quadrature wheel encoders tracked via Raspberry Pi 5 GPIO interrupts (`lgpio` / `gpiozero`).
    - **User Interface:** Onboard LCD screen (procedural eye animations), 2 physical hardware trigger buttons (Button A: Track/Follow, Button B: Return/SOS), and an onboard audio output interface (speaker/amplifier for emergency vocal broadcast).

2. Operational State Machine

State ID	State Name	Entry Trigger	Active Software Behaviors	LCD & Audio Status
S0	IDLE / STANDBY	System boot or mission completion	Motor controllers locked (`cmd_vel` = 0); LiDAR spinning in standby; waiting for GPIO interrupt.	**LCD:** Procedural looping eye animations (random blinks, saccades, idle looking).**Audio:** Silent.
S1	FOLLOW & MAP (Outbound)	Button 1 pressed	**1. Human Tracking:** 2D LiDAR clusters leg/torso signatures and maintains target follow distance.**2. SLAM & Odometry:** Encoders + LiDAR update 2D occupancy grid (`slam_toolbox`) and drop (x, y, theta) pose "breadcrumbs" along the trail.	**LCD:** Focused/tracking eye state (eyes shift in the direction of the tracked human's angle).**Audio:** Short confirmation chime on start.

State ID	State Name	Entry Trigger	Active Software Behaviors	LCD & Audio Status
S2	BACKTRACK (Lost Hiker Return)	Button 2 pressed (Single press)	**1. Path Reversal:** Loads recorded breadcrumb waypoints in reverse (N down to 0).**2. Dynamic Obstacle Avoidance:** Uses live LiDAR scans to navigate around newly fallen branches, rocks, or shifted terrain while hugging the original global breadcrumb path back to (0, 0).	**LCD:** Determined/scanning eye animation.**Audio:** Periodic status beep.
S3	EMERGENCY SOS & SEEK HELP	Button 2 pressed (Long press / SOS trigger)	**1. Autonomous Backtrack:** Executes full reverse path navigation with dynamic obstacle avoidance back toward the trailhead/base.**2. Active Human Seeking:** Runs LiDAR human-detection clustering while backtracking to spot nearby hikers/responders.**3. Distress Broadcast:** Loops vocal audio alert ("Please help! A hiker needs assistance!") and approaches/signals detected humans to lead them back.	**LCD:** High-contrast flashing SOS / wide alert eyes.**Audio:** Continuous looping "Please help" vocal distress broadcast.

3. Functional Software Requirements Sheet

A. Perception & Human Tracking (`lidar_human_follower`)

Req ID	Requirement Name	Technical Specification & Behavior	Target Node / Package
FR-PER-01	LiDAR Driver & Filtering	Ingest 360-degree `sensor_msgs/LaserScan` packets at 10 Hz (230,400 baud). Apply range gate (0.15 m to 8.0 m) and NaN/infinity filtering.	`ldlidar_stl_ros2 (/sca…`

Req ID	Requirement Name	Technical Specification & Behavior	Target Node / Package
FR-PER-02	Euclidean Leg/Torso Clustering	Segment raw `/scan` points into distinct clusters based on distance jump threshold ($\Delta r < 0.15\text{ m}$) and filter clusters matching human width profiles (0.10 m to 0.50 m).	`human_tracker_node`
FR-PER-03	Target Lock & Centroid Tracking	Upon Button 1 press, lock onto the nearest valid human cluster in the front arc (-60° to $+60^\circ$). Track cluster centroid (r, θ) across consecutive frames using nearest-neighbor/Kalman association.	`human_tracker_node`
FR-PER-04	Active Rescue Human Detection (SOS Mode)	During S3 (Emergency SOS) backtracking, scan the full 360-degree field for non-static clusters matching human geometry (FR-PER-02) outside the immediate obstacle collision zone to identify potential rescuers.	`human_tracker_node(/so…`

B. Odometry, Localization & 2D SLAM (`slam_and_breadcrumbs`)

Req ID	Requirement Name	Technical Specification & Behavior	Target Node / Package
FR-SLAM-01	Differential Drive Encoder Odometry	Read quadrature ticks from Left and Right motors via hardware GPIO interrupts at 50 Hz. Compute linear velocity v, angular velocity ω, and dead-reckoning pose (x, y, θ). Publish	`diff_drive_odom_node`

Req ID	Requirement Name	Technical Specification & Behavior	Target Node / Package
		`nav_msgs/Odometry` and `odom` $\rightarrow$ `base_link` TF.	
FR-SLAM-02	2D LiDAR-Inertial/Odometry SLAM	Fuse wheel odometry (`odom` $\rightarrow$ `base_link`) with 2D `/scan` laser matching to generate a drift-corrected `map` $\rightarrow$ `odom` transform and a live 2D `nav_msgs/OccupancyGrid`.	`slam_toolbox`(async online mode)
FR-SLAM-03	Breadcrumb Trail Recorder	While in S1 (Follow Mode), append the robot's localized map-frame pose (x, y, θ, timestamp) to an in-memory and disk-backed breadcrumb stack every time Euclidean displacement exceeds 0.25 m or heading changes by more than 15°.	`breadcrumb_manager_node`

C. Motion Control & Backtrack Navigation (`navigation_control`)

Req ID	Requirement Name	Technical Specification & Behavior	Target Node / Package
FR-NAV-01	PD Human-Following Controller	In S1, compute `geometry_msgs/Twist` (`cmd_vel`) using a Proportional-Derivative (PD) controller to maintain a target standoff distance (1.0 m) and zero angular error ($\theta = 0$) relative to the locked human centroid.	`rover_controller_node`

Req ID	Requirement Name	Technical Specification & Behavior	Target Node / Package
FR-NAV-02	Reverse Waypoint Follower	In S2 and S3, pop recorded breadcrumb poses in LIFO (Last-In, First-Out) order and steer toward each waypoint sequentially using Pure Pursuit or PID heading control until reaching the origin $(0, 0)$.	`backtrack_navigator_no…`
FR-NAV-03	Dynamic Local Obstacle Avoidance	While backtracking along the breadcrumb trail, continuously check a forward safety sector (-45° to $+45^\circ$, range $< 0.60\text{ m}$). If a newly placed obstacle blocks the recorded path, execute local Vector Field Histogram (VFH) / Dynamic Window (DWA) steering around the obstruction before rejoining the next unobstructed breadcrumb.	`backtrack_navigator_no…`
FR-NAV-04	Closed-Loop Motor PWM Driver	Subscribe to `/cmd_vel`, convert (v, ω) into left/right wheel target RPMs, and apply PID PWM control using encoder feedback to compensate for uneven trail friction or battery voltage sag.	`motor_driver_node`

D. Human-Robot Interaction: LCD, Buttons & SOS Audio (`hri_subsystem`)

Req ID	Requirement Name	Technical Specification & Behavior	Target Node / Package
FR-HRI-01	Procedural LCD Eye Animator	Render a 30+ FPS procedural eye animation loop on the LCD screen. Support randomized blink intervals ($2\text{--}6\text{ sec}$), random idle pupil gaze	`lcd_eye_animator_node`

Req ID	Requirement Name	Technical Specification & Behavior	Target Node / Package
		shifts in S0, directional gaze tracking matched to the human's LiDAR angle θ in S1, and flashing distress graphics in S3.	
FR-HRI-02	Debounced GPIO Button Handler	Poll or attach edge-detection interrupts to the two hardware buttons with a 50 ms software debounce. Publish state transition events (`/mode_command`) to the central state machine.	`gpio_button_node`
FR-HRI-03	Emergency SOS Vocal Broadcast	Upon entering S3 (Emergency SOS), spawn a non-blocking audio playback thread (`aplay` / `pygame.mixer`) that loops a pre-recorded `.wav` distress message ("Please help! Emergency on trail!"). Increase volume/cadence when FR-PER-04 detects a nearby human.	`sos_audio_node`

5. Non-Functional & Performance Requirements

ID	Category	Requirement Specification
NFR-01	100% Offline Autonomy	All SLAM, LiDAR clustering, breadcrumb storage, LCD rendering, and audio synthesis must run locally on the Raspberry Pi 5 with zero external Wi-Fi, cellular, or cloud API dependencies.

ID	Category	Requirement Specification
NFR-02	Real-Time Scan-to-Actuation Latency	End-to-end processing time from `/scan` message receipt to updated `/cmd_vel` motor output must stay below 100 ms (10 Hz control loop minimum) to prevent losing fast-moving hikers or colliding with sudden obstacles.
NFR-03	Crash-Safe Breadcrumb Persistence	Logged breadcrumb coordinates must be flushed to a local `.csv` or `.json` file on the SD card every 5 waypoints so the trail history survives an unexpected node restart or brownout.
NFR-04	Safety Deadman Timeout	If `/cmd_vel` or `/scan` stops publishing for more than 500 ms (e.g., sensor disconnect or node crash), `motor_driver_node` must immediately cut PWM output to 0 and trigger a visual warning on the LCD.